ECC 模块联动架构：Skill / Agent / Command 统一贯穿机制

核心发现

ECC（everything-claude-code）的 skill、agent、command **并非自动一一对应**，而是**通过多层架构和约定实现语义对齐。**

一、三个组件的角色分工

组件	位置	性质	用户触发方式
Skill	`skills/<name>/SKILL.md` (136个)	领域知识文档，Agent 的知识库	不可直接调用，由 Agent 内部引用
Agent	`agents/<name>.md` (30个)	可被委托的子任务执行者	不可直接调用，由 Command 或其他 Agent 触发
Command	`commands/<name>.md` (60个)	用户入口，用户通过 `/<name>` 触发	用户直接调用

二、组件间的引用关系

```
用户 /plan
  └→ Command: commands/plan.md
        └→ 内部说明 "invokes the planner agent"
              └→ Agent: agents/planner.md
                    └→ 知识自包含（无外部 skill 引用）
                          如需外部 skill，由 tdd-guide 等 agent 引用
                                └→ Skill: skills/tdd-workflow/SKILL.md
```

实际的引用机制：

- **Command → Agent**：Command 文件正文中写明 "This command invokes the `planner` agent"，无自动化脚本，纯文字约定。
- **Agent → Skill**：Agent 文件中用 `skill: <skill-name>` 内部标记引用对应 Skill。
- **Agent → Agent**：`/orchestrate` 命令形成链式调用：`planner → tdd-guide → code-reviewer → security-reviewer`。

三、Manifest 系统——安装层面的绑定

ECC 设计了 **三层 Manifest 系统**，在安装时将相关组件打包：

Tier 1：`manifests/install-modules.json`（模块定义）

定义 15 个模块，每个模块包含一组相关路径：

```
{
  "agents-core": {
    "paths": [".agents", "agents", "AGENTS.md"]
  },
  "workflow-quality": {
    "paths": [
      "skills/tdd-workflow",
      "skills/eval-harness",
      "commands/tdd.md",
      "commands/e2e.md",
      "agents/tdd-guide.md",
      "agents/e2e-runner.md"
    ]
  }
}
```

关键洞察：`workflow-quality` 模块同时包含 `tdd` 的 skill、command、agent——这就是安装层面的"组件对齐"。

Tier 2：`manifests/install-profiles.json`（安装配置）

将模块组合成 5 个安装配置（core / developer / security / research / full）。

Tier 3：`manifests/install-components.json`（命名组件）

提供细粒度安装点：

```
agent:planner        → agents-core
agent:tdd-guide       → agents-core
skill:tdd-workflow    → workflow-quality
command:tdd          → workflow-quality
capability:security  → security 模块
```

四、命名约定（约定的来源）

Command	Agent	Skill
`commands/plan.md`	`agents/planner.md`	无独立 Skill（知识自包含）
`commands/tdd.md`	`agents/tdd-guide.md`	`skills/tdd-workflow/`
`commands/e2e.md`	`agents/e2e-runner.md`	`skills/e2e-testing/`
`commands/code-review.md`	`agents/code-reviewer.md`	`skills/code-review/`

没有自动化强制，完全靠：

1. 文件名约定（`plan` ↔ `planner` ↔ `planning`）
2. Manifest 模块打包时手动对齐
3. Agent 文件内部的 `skill:` 引用

五、Skill 的分类（来源追踪）

ECC 在 `docs/SKILL-PLACEMENT-POLICY.md` 中定义了 Skill 的四种来源：

类型	路径	说明
Curated	`skills/` (仓库内)	ECC 自带
Learned	`~/.claude/skills/learned/`	`/learn` 命令从对话中提取
Imported	`~/.claude/skills/imported/`	外部安装
Evolved	`~/.claude/homunculus/evolved/skills/`	本地本能生成

六、Rules——贯穿所有组件的全局约束

`rules/` 目录下的规则（common/ + per-language/）通过 `CLAUDE.md` 引用后，**始终加载**，不经过任何 agent/command/skill 路径，对所有组件生效。

七、架构总结图（Command/Agent/Skill 联动）

```
用户输入
  │
  ▼
┌──────────────┐
│  Command     │  "invokes the planner agent" (纯文字)
│ (/plan, /tdd) │
└──────────────┘
       │ spawn Agent(name="planner", tools=[...])
       ▼
┌──────────────┐
│   Agent      │
│  (planner)   │  知识自包含 OR "skill: tdd-workflow"
└──────────────┘
       │ skill: xxx
       ▼
┌──────────────┐
│    Skill     │  流程型（如 tdd-workflow）由多 Agent 复用
│ (领域知识库)  │  规划型（如 planner）知识通常自包含
└──────────────┘

Manifest 在安装时将相关组件打包进同一模块
Rules (rules/) ──────→ 全局始终加载
Hooks (hooks/) ──────→ 事件触发，跨所有组件
AGENTS.md ──────────→ 编排说明（非自动化）
```

九、以 `planner` 为例：完整联动链路解析

以下通过 `/plan` 命令的完整生命周期，展示三个组件如何实际联动。

9.1 用户触发：Command 层

文件： `commands/plan.md`

```
---
description: Restate requirements, assess risks, and create step-by-step
implementation plan. WAIT for user CONFIRM before touching any code.
---
```

核心内容：

- 用户在 Claude Code 中输入 `/plan`
- Command 文件正文中明确声明：

  > This command invokes the **planner** agent to create a comprehensive implementation plan before writing any code.

- 说明下一步应该调用哪个 agent（纯文字约定，无自动化脚本）
- 文档底部标注来源引用：

  ```
  ## Related Agents
  This command invokes the `planner` agent provided by ECC.
  For manual installs, the source file lives at: `agents/planner.md`
  ```

关键字段说明：

字段	值	含义
`description`	完整功能描述	Claude Code 在用户输入 `/` 时展示为提示
正文第一句	"invokes the planner agent"	**唯一的联动契约**，告知用户和框架接下来调用谁

9.2 委托执行：Agent 层

文件： `agents/planner.md`

```
---
name: planner
description: Expert planning specialist for complex features and refactoring...
```

```
tools: ["Read", "Grep", "Glob"]
model: opus
---
```

Agent 的结构：

1. **Frontmatter 元信息**：声明名称、描述、可用工具（`Read/Grep/Glob`）、使用模型（`opus`）
2. **角色定义**（`Your Role`）：规划专家的职责
3. **规划流程**（`Planning Process`）：4步方法论（需求分析→架构审查→步骤拆分→实现顺序）
4. **标准输出格式**（`Plan Format`）：规定的 Markdown 输出模板
5. **完整示例**（`Worked Example: Adding Stripe Subscriptions`）：附完整示例
6. **最佳实践** + **反模式检查清单**

与 Command 的对应：

Command `/plan` 说	Agent `planner` 做的事
"MUST WAIT for user CONFIRM before code"	在流程末尾停住，等用户确认
"create step-by-step implementation plan"	提供 Phase/Step 层级拆解
"assess risks"	`Risks & Mitigations` 章节

9.3 知识沉淀：Skill 层（可选）

重要发现：`planner` agent 没有引用外部 Skill！

与 `tdd-guide` agent 使用 `skill: tdd-workflow` 不同，`planner` agent 的所有领域知识（规划方法论、格式模板、反模式清单）都**直接内嵌在 agent 文件中。**

这是 ECC 中两种 agent 设计方式的体现：

类型	示例	Skill 依赖
知识密集型	`planner`、`architect`	无外部 Skill，知识自包含
流程指导型	`tdd-guide`、`e2e-runner`	引用 `skill: tdd-workflow`、`skill: e2e-testing`

原因分析：

- `planner` 需要根据用户项目的**具体代码**做规划（要 `Read/Grep/Glob`），知识必须实时生成
- `tdd-guide` 指导的是**标准化流程**，Skill 中存储的 Playwright 语法、测试框架细节是稳定的

9.4 安装层面绑定：Manifest 中的体现

```
install-components.json:
  agent:planner  →  agents-core 模块  →  包含 agents/ 目录

install-modules.json:
```

```
  agents-core.paths = [".agents", "agents", "AGENTS.md"]
  commands-core.paths = ["commands"]   ← plan.md 在这里
```

Manifest 的实际效果：

- 安装 `core` profile → 自动获得 `commands/plan.md` 和 `agents/planner.md`
- 安装 `agent:planner` 组件 → 从 `agents-core` 安装整个 `agents/` 目录

注意：`plan.md` command 和 `planner` agent 不在同一个 manifest 模块中打包，它们通过 `core` profile 同时被安装，而非像 `workflow-quality` 那样显式对齐。

9.5 完整数据流图

```
[用户输入 /plan]
        │
        ▼
┌──────────────────────────────────────┐
│  Command: commands/plan.md           │
│  描述: Restate requirements, assess...  │
│  正文: "invokes the planner agent"    │
│  (纯文字约定，无自动化)               │
└──────────────────────────────────────┘
                │
                │ Claude Code 解析 description
                │ spawn Agent(name="planner", tools=[...])
                ▼
┌──────────────────────────────────────┐
│  Agent: agents/planner.md            │
│  model: opus                         │
│  tools: [Read, Grep, Glob]           │
│  → 读取项目代码，生成 plan            │
│  → 输出格式化的 Markdown Plan         │
│  → 等待用户 CONFIRM                  │
└──────────────────────────────────────┘
                │
                │ 用户确认 "yes"
                ▼
┌──────────────────────────────────────┐
│  后续调用链 (AGENTS.md 中定义)        │
│  planner → tdd-guide → code-reviewer  │
│  (tdd-guide 会引用 skill: tdd-workflow) │
└──────────────────────────────────────┘
```

9.6 AGENTS.md 中的编排定义

`AGENTS.md` 是另一个重要文件，它是 agent 之间的**编排层**：

```
Complex feature requests → planner
Bug fix or new feature   → tdd-guide
```

```
Code just written        → code-reviewer
```

这里的编排关系是**指导性文档**，告诉用户/系统在不同场景下应该使用哪个 agent，但同样**不是自动化脚本**。

9.7 小结：`planner` 案例的关键启示

1. **Command 不等于 Agent**：`/plan` 是用户入口，`planner` 是任务执行者，由 Command 正文中的文字约定联动
2. **Agent 可以有外部 Skill，也可以没有**：`planner` 自包含知识；`tdd-guide` 引用 `skill: tdd-workflow`
3. **Manifest 不强制对齐**：两个组件可以在不同模块中，通过 profile 同时安装实现间接对齐
4. **所有联动都是"约定"而非"代码"**：ECC 的模块间引用全是文字说明 + 目录路径，没有硬编码函数调用
5. **AGENTS.md 是编排说明文档**：不是配置，但提供了 agent 之间的逻辑关系

十、以 `tdd` 为例：Command → Agent → Skill 完整三角链路

以下通过 `/tdd` 命令，展示**三者完整联动**的完整生命周期。与 `planner` 不同，`tdd` 真正涉及 Command → Agent → Skill 三层全部联动。

10.1 用户触发：Command 层

文件： `commands/tdd.md`

```
---
description: Enforce test-driven development workflow. Scaffold interfaces,
generate tests FIRST, then implement minimal code to pass. Ensure 80%+ coverage.
---
```

核心内容：

- 用户输入 `/tdd`，Command 文件激活
- 正文开篇即声明联动关系：

  > This command invokes the **tdd-guide** agent provided by ECC.

- 文档末尾显式标注 Skill 来源（**这是 `planner` 没有的细节**）：

  ```
  ## Related Agents
  This command invokes the `tdd-guide` agent provided by ECC.
  The related `tdd-workflow` skill is also bundled with ECC.

  For manual installs, the source files live at:
  - agents/tdd-guide.md
  - skills/tdd-workflow/SKILL.md
  ```

`/tdd` 与 `/plan` 的关键区别： Command 层直接告知用户 Skill 也被包含，而 `plan` Command 只提 agent。

10.2 委托执行：Agent 层

文件： `agents/tdd-guide.md`

```
---
name: tdd-guide
description: Test-Driven Development specialist enforcing write-tests-first...
tools: ["Read", "Write", "Edit", "Bash", "Grep"]
model: sonnet
---
```

Agent 结构（6模块）：

模块	内容	对应 Skill
`Your Role`	TDD 专家职责定义	—
`TDD Workflow`	Red-Green-Refactor 5步流程	—
`Test Types Required`	Unit/Integration/E2E 分类表	—
`Edge Cases You MUST Test`	8类边界情况清单	—
`Test Anti-Patterns`	5类反模式	—
`Quality Checklist`	9项检查项	—
`v1.8 Eval-Driven TDD Addendum`	评估驱动开发扩展	—

引用 Skill 的位置（第80行）：

```
For detailed mocking patterns and framework-specific examples,
see `skill: tdd-workflow`.
```

关键发现：Skill 引用位置在文件末尾，不是开头——说明 Skill 是**可选的详细参考**，而非执行的必要前提。

工具集对比：

Agent	tools	model
`planner`	`[Read, Grep, Glob]`	`opus`
`tdd-guide`	`[Read, Write, Edit, Bash, Grep]`	`sonnet`

- `planner` 只读不写（分析规划），`sonnet` 足够
- `tdd-guide` 需要写测试文件 + Bash 执行测试，`sonnet` 够用但工具更多

10.3 知识沉淀：Skill 层

文件： `skills/tdd-workflow/SKILL.md`

```
---
name: tdd-workflow
description: Use this skill when writing new features, fixing bugs, or refactoring
code...
origin: ECC
---
```

Skill 结构（与 Agent 的分工）：

Skill 内容	Agent 内容	分工说明
User Journey 编写规范	TDD 流程框架	Agent 负责流程，Skill 补充需求捕获
RED/GREEN/REFACTOR 7步	5步简化版	Skill 更详细，Agent 做概要
Jest/Vitest 单元测试代码模板	—	**Skill 独有**，Agent 引用
API Integration 测试模板	—	**Skill 独有**
Playwright E2E 测试模板	—	**Skill 独有**
Supabase/Redis/OpenAI Mock 模板	—	**Skill 独有**，框架级细节
测试文件组织结构	—	**Skill 独有**
CI/CD 集成示例	—	**Skill 独有**
Checkpoint Git 提交规范	—	**Skill 独有**，ECC 特有实践

Skill 的两个核心原则（第19-28行）：

```
Tests BEFORE Code
Coverage Requirements: 80%+ minimum, all edge cases, error scenarios
```

RED Gate 强制机制（第98-118行）：

Skill 中有**强制性的 RED 验证步骤**，这段逻辑在 Agent 中被简化引用：

```
// Skill 原文（第103行）：
This step is mandatory and is the RED gate for all production changes.

// 验证路径：
// 1. Runtime RED → 测试编译通过 + 执行失败
// 2. Compile-time RED → 编译失败本身即 RED 信号
// 3. 两者都必须是由业务逻辑 bug 导致，而非语法错误
```

10.4 Manifest 中的完整打包

`workflow-quality` 模块——TDD 三组件的显式对齐：

```
{
  "id": "workflow-quality",
  "paths": [
    "skills/tdd-workflow",
    "skills/eval-harness",
    "commands/tdd.md",
    "commands/e2e.md",
    "agents/tdd-guide.md",
    "agents/e2e-runner.md"
  ]
}
```

三层 Manifest 联动关系：

```
install-components.json:
  agent:tdd-guide    → agents-core        → [agents/]
  command:tdd       → workflow-quality   → [commands/tdd.md]
  skill:tdd-workflow → workflow-quality   → [skills/tdd-workflow]

install-modules.json:
  workflow-quality.paths = [
    "skills/tdd-workflow",   ← Skill
    "commands/tdd.md",        ← Command
    "agents/tdd-guide.md"     ← Agent
  ]
   (agents-core 通过 dependency 引入 agents/)

install-profiles.json:
  developer profile 包含 workflow-quality
  security profile   包含 workflow-quality
  full profile       包含 workflow-quality
```

关键差异：`planner` vs `tdd` 的 Manifest 对齐程度：

对比项	`planner`	`tdd`
Command/Agent/Skill 是否在同一模块	**否**	**是**（workflow-quality）
Command 说明中是否提 Skill	否	**是**（"tdd-workflow skill is also bundled"）
Agent 是否引用外部 Skill	否	**是**（`skill: tdd-workflow`）
Manifest 模块是否显式命名	否	**是**（workflow-quality）

10.5 完整数据流图（TDD 三角链路）

```
[用户输入 /tdd]
        │
        ▼
┌──────────────────────────────────────┐
│  Command: commands/tdd.md            │
│  description: Enforce TDD workflow...  │
│  正文: "invokes the tdd-guide agent"   │
│  末尾注: "tdd-workflow skill is bundled" │
└──────────────────────────────────────┘
               │
               │ spawn Agent(name="tdd-guide", tools=[...])
               ▼
┌──────────────────────────────────────┐
│  Agent: agents/tdd-guide.md          │
│  tools: [Read, Write, Edit, Bash, Grep]  │
│  model: sonnet                       │
│                                      │
│  → 执行 Red-Green-Refactor 循环       │
│  → 末尾引用: "see skill: tdd-workflow"  │
└──────────────────────────────────────┘
               │
               │ 读取 skill: tdd-workflow
               ▼
┌──────────────────────────────────────┐
│  Skill: skills/tdd-workflow/SKILL.md  │
│                                      │
│  提供:                                │
│  - Jest/Playwright 代码模板           │
│  - Mock patterns (Supabase/Redis/OpenAI)│
│  - RED gate 强制验证逻辑              │
│  - CI/CD 集成示例                     │
│  - Checkpoint Git 提交规范            │
└──────────────────────────────────────┘
               │
               │ 执行测试 (Bash 工具)
               ▼
        [测试结果反馈]
```

10.6 TDD vs Planner：设计模式对比总结

维度	`planner`	`tdd`
Agent 类型	知识密集型	流程指导型
外部 Skill	无	`skill: tdd-workflow`
Skill 位置	—	Agent 末尾引用（非前置）
Manifest 对齐	间接（core profile）	**显式**（workflow-quality）
Command 中提 Skill	否	是

维度	`planner`	`tdd`
Agent 内嵌内容	完整规划方法论	TDD 流程框架（详细靠 Skill）
模型选择	`opus`（复杂推理）	`sonnet`（执行型）
工具集	只读	读/写/执行

设计模式总结：

- **知识密集型 Agent**（planner）：自包含所有知识 → 无外部 Skill 依赖 → 用 `opus` 做深度推理
- **流程指导型 Agent**（tdd-guide）：主流程内嵌 + 细节靠 Skill → `skill:` 引用在末尾 → 用 `sonnet` 做执行

10.7 小结：`tdd` 案例的关键启示

1. **三角链路完整存在**：`/tdd` → `tdd-guide` → `tdd-workflow` 三层全部参与联动
2. **Skill 是 Agent 的"详细手册"**：Agent 给出流程框架，Skill 填充代码模板和框架细节
3. **Skill 引用位置有讲究**：在 Agent 末尾而非开头——Skill 是"详细参考"而非"执行前提"
4. **Manifest 可以显式对齐**：workflow-quality 模块将三者显式打包，优于 planner 的间接对齐
5. **Command 层次可以宣传 Skill**：`tdd` Command 末尾直接告诉用户 Skill 也被包含，planner 则没有

十一、关键设计原则

1. **组件解耦但语义对齐**：Command/Agent/Skill 是独立文件，通过命名约定和文字引用互联，非硬编码依赖。
2. **Manifest 是安装绑定层**：组件间的关系在安装时通过 Manifest 打包确定，而非运行时动态发现。
3. **Skill 是底层知识单元**：最细粒度，可被多个 Agent 复用；Agent 是任务执行单元；Command 是用户入口。
4. **无自动强制**：所有对应关系靠文档约定（`RULES.md` / `CLAUDE.md`）和人类规范。